

# WMS Projesi — Faz 1-4 Uygulama Dokümantasyonu

Proje: WMS Çoklu Lokasyon & Lokalizasyon Projesi

Kapsam: Faz 1 – Faz 4 (15 iş isteri)

Teknoloji: Java 21, Spring Boot 3.x, Spring Data JPA, PostgreSQL, Redis, Kafka, Keycloak, Flyway

Tarih: 10 Temmuz 2026

## 1. Genel Bakış

Proje, çoklu firma (multi-tenant) ve çoklu lokasyon destekleyen bir Depo Yönetim Sistemi'nin lokalizasyon, finans ve entegrasyon altyapısını 4 fazda hayata geçirmiştir. Uygulama mikroservis mimarisıyla bölünmüştür:

| Servis                       | Sorumluluk                                                                            | Ana Fazlar |
|------------------------------|---------------------------------------------------------------------------------------|------------|
| wms-core-service             | Organizasyon hiyerarşisi, tenant context, auth, workflow, dinamik UI, geo master data | Faz 1, 2   |
| wms-localization-service     | Dil/çeviri, format konfigürasyonu, adres şablonları, JSONB adres doğrulama            | Faz 1, 2   |
| wms-finance-service          | Para birimi, kur çevrimi, TCMB entegrasyonu, vergi motoru                             | Faz 3      |
| wms-billing-service          | Çoklu para birimli faturalama, kur kilidi, kur farkı                                  | Faz 4      |
| wms-integration-service      | ERP adaptörleri, Outbox altyapısı, entegrasyon izleme                                 | Faz 4      |
| wms-inventory / wms-outbound | Stok ve sevkiyat süreçleri (workflow ve entegrasyon tüketicisi)                       | Faz 2, 4   |

Faz bağımlılıkları: Faz 1 → (Faz 2, Faz 3) → Faz 4.

## 2. Faz 1 — Çekirdek Sistem & Çoklu Lokasyon Temeli

Kapsam: İş İsterleri 1, 6, 7, 13 · Servisler: wms-core-service, wms-localization-service

### 2.1 Organizasyon Hiyerarşisi ve Veri Modeli

Hiyerarşik veri modeli kuruldu: **Organization** → **Company** → **Region** → **Location** (Depo) → **Zone** → **StorageLocation**. Kullanıcı yetkilendirmesi **User**, **Role** ve şirket/lokasyon bazlı erişimi tanımlayan **UserAccess** entity'leriyle modellendi. Tüm tablolar Flyway migration'larıyla versiyonlandı (core-service: 22 migration).

### 2.2 Multi-Tenancy ve Context İzolasyonu

- API isteklerinde **X-Active-Company-ID** ve **X-Active-Location-ID** header'ları **TenantContextFilter** tarafından okunur, JWT'deki **UserAccess** yetkileriyle doğrulanır ve request-scoped **TenantContextHolder** (ThreadLocal) içinde saklanır.
- Tenant kapsamındaki entity'ler **BaseScopedEntity**'den türer; **TenantEntityListener** yazma anında company/location bilgisini otomatik doldurur.

- `TenantFilterAspect` sorgulara tenant filtresini otomatik uygular; istisnai durumlar `@IgnoreTenantFilter` ile işaretlenir.

## 2.3 Kimlik Doğrulama (Keycloak)

Kimlik doğrulama Keycloak'a devredildi: `KeycloakAuthService` (login/token), `KeycloakAdminService` (kullanıcı provisioning) ve `KeycloakJwtAuthenticationConverter` (rol/claim eşleme) yazıldı. Tüm servislerde ortak `SecurityConfig` + JWT doğrulama standardı uygulandı.

## 2.4 Zaman Yönetimi — UTC Standardı

- Tüm zaman kolonları PostgreSQL `TIMESTAMPTZ` (UTC) olarak saklanır; `JacksonConfig` API JSON çıktısını ISO 8601 UTC formatına zorlar.
- `TimezoneService` lokasyonun IANA timezone bilgisine göre sunucu taraflı dönüşüm yapar; `DateRangeUtcQueryHelper` yerel tarih aralığı sorgularını UTC aralığına çevirir.
- `TransactionLogReportService` raporlarda zamanları aktif lokasyonun saat diliminde sunar.

## 2.5 Tarih/Saat ve Sayı Format Yönetimi

wms-localization-service içinde `CountryFormatConfig` (ülke bazlı tarih maskesi, ondalık/binlik ayıraç) ve `LocationFormatOverride` (lokasyon bazlı ezme) entity'leri kuruldu.

`FormatConfigController` aktif formatları (`ActiveFormatResponse`) arayüze API ile sunar;

`ReportFormatterService` sunucu taraflı rapor formatlamasını yapar.

## 2.6 Hiyerarşik Adres Master Data

`Country` → `StateProvince` → `City` → `District` → `Neighborhood` zinciri kuruldu. `GeoAdminController` master data CRUD ve toplu import uçlarını, `AddressController` bağımlı (cascading) seçim API'lerini sunar. `AddressMasterValidationService` hiyerarşi tutarlılığını doğrular.

# 3. Faz 2 — Dinamik UI & Parametrik İş Akışı

Kapsam: İş İsterleri 3, 5, 12 · Servisler: wms-core-service, wms-localization-service

## 3.1 Workflow Engine (Lokasyon Süreç Konfigürasyonu)

- Master süreç tanımları `ProcessDefinition` / `ProcessStepDefinition`; lokasyon bazlı sıra (sequence) ve zorunluluk ayarları `LocationProcessConfig` / `LocationProcessStepConfig` ile yönetilir. Yönetim API'si: `ProcessConfigController`.
- `WorkflowValidatorService` bir adımın çalıştırılabilirliğini (önceki zorunlu adımlar tamam mı?) kontrol eder. İş servisleri `@CheckWorkflowStep` anotasyonu ile işaretlenir; `WorkflowAspect` (AOP) doğrulamayı çağrı öncesi otomatik uygular.
- Adım konfigürasyonları Redis'te cache'lenir; `WorkflowCacheEvictionListener` konfigürasyon değişikliğinde cache'i düşürür. Değişiklikler `ConfigurationAuditLog` + audit listener ile izlenir.
- Kritik konfigürasyon değişiklikleri için onay akışı eklendi: `ApprovalRequest`, `ApprovalController`, `ApprovalRequiredException`.

## 3.2 Dynamic UI Engine (Ekran Alan Yönetimi)

- Ekran ve alan tanımları: `Screen`, `ScreenField`. Davranış kuralları `FieldBehaviorRule` ile lokasyon/rol/operasyon tipi bağlamlarına göre öncelik (priority) mantığıyla tanımlanır; `UiContextFactory` aktif bağlamı üretir, `DynamicUiService` kuralları çözümleyip arayüze JSON şeması (`ResolvedScreenDto`) döner.

- Tablo kolonları için paralel yapı kuruldu: `TableColumnDef`, `ColumnBehaviorRule`, kullanıcı bazlı `UserTablePreference`; çözümleme `DynamicTableUiService`, yönetim `TableColumnAdminController` ve `UiRuleManagementController` ile yapılır.
- Sunucu tarafı doğrulama: POST/PUT uçları `@ValidateDynamicForm` ile işaretlenir; `DynamicFormValidationAspect` aktif kuralları uygulayıp ihlalde `DynamicValidationException` (alan bazlı hata listesi) fırlatır.

### 3.3 JSONB Adres Şablonları ve Regex Doğrulama

- Adres verisi wms-localization-service'te `Address` entity'sinde JSONB olarak saklanır.
- Ülke bazlı şablonlar `CountryAddressTemplate` / `AddressTemplateField` ile tanımlanır: alan sırası, zorunluluk ve regex desenleri. `AddressValidationService` JSONB içeriği bu kurallara göre doğrular; `AddressTemplateAdminController` şablon kopyalama ve yeniden sıralama uçlarını sunar.
- Görüntüleme için Strategy Pattern uygulandı: `AddressFormatterStrategy` arayüzünü `TurkeyAddressFormatterStrategy` ve `UsAddressFormatterStrategy` uygular; `AddressFormatterService` ülkeye göre stratejiyi seçer.

## 4. Faz 3 — Finansal Altyapı, Döviz & Vergi Motoru

Kapsam: İş İsterleri 8, 9, 10, 14, 15 · Servis: wms-finance-service (13 migration)

### 4.1 Çoklu Para Birimi Altyapısı

- `FinancialTransaction` tutarları `original_amount`, `exchange_rate`, `converted_amount` kırımlıyla `BigDecimal` olarak saklar; kullanılan kurun geçmiş tarihli olması `fallback_rate_used` bayrağıyla izlenir.
- `CurrencyConversionService`: tüm yuvarlamalarda Banker's Rounding (`RoundingMode.HALF_EVEN`); kur bulunamadığında en fazla 5 gün (`MAX_FALLBACK_DAYS = 5`) geçmişe fallback; çapraz kur ve ters kur (inversion) hesaplama desteği.
- Şirket ve lokasyon bazlı para birimi ayarları: `CompanyCurrencySetting`, `LocationCurrencySetting`, `CurrencyResolverService`.

### 4.2 Müşteri Döviz Kısıtları

`FinanceCustomer` kartında varsayılan para birimi ve `CustomerPermittedCurrency` ile izin verilen döviz listesi tanımlanır. Sipariş (`OrderController` / `OrderService`) ve sözleşme (`ContractController`) uçlarında `CustomerCurrencyValidator` ve `ContractCurrencyValidator` API seviyesinde doğrulama yapar; ihlalde `InvalidCustomerCurrencyException` döner.

### 4.3 Döviz Kuru Yönetimi ve TCMB Entegrasyonu

- `TcmbRateSyncJob` her gün 15:35'te (Europe/Istanbul) `@Scheduled(cron = "0 35 15 *")` ile TCMB XML servisinden kurları çeker; `TcmbXmlParser` ayrıştırır, `TcmbRateUpsertService` `ExchangeRate` tablosuna upsert eder. Uygulama açılışında `TcmbStartupSyncRunner` eksik günleri tamamlar.
- Manuel kur girişleri `ManualRateService` üzerinden yapılır ve her değişiklik `ExchangeRateAuditLog`'a (eski/yeni değer, kullanıcı, zaman) yazılır.
- Kurlar Redis'te cache'lenir (`ExchangeRateCacheService`); sorgu uçları `ExchangeRateController` ve `CurrencyRateLookupController`.

### 4.4 Versiyonlanmış Vergi Oranları

`TaxRate` kayıtları geçerlilik tarih aralıklarıyla versiyonlanır; `TaxRateVersioningService` tarih çakışmalarını engeller (`TaxRateConflictException`) ve önceki versiyonu otomatik kapatır. Tüm değişiklikler `TaxRateAuditLog`'a yazılır. `TaxResolutionService` geçerli oranı Ülke → Lokasyon → Müşteri → Ürün öncelik zinciriyle dinamik çözümler.

## 4.5 Tax Engine (Strategy Pattern)

- `TaxCalculationStrategy` arayüzü üç stratejiyle uygulandı: `TaxInclusiveStrategy` (vergi dahil), `TaxExclusiveStrategy` (vergi hariç), `CompoundTaxStrategy` (katmanlı, örn. ÖTV + KDV).
- `TaxStrategyFactory` stratejiyi dinamik seçer; `TaxEngineService` satır bazlı hesaplama yapar ve matrah/oran/hesap detaylarını izlenebilirlik için `TaxCalculationLog` tablosuna (`TaxCalculationLogWriter`) yazar.

## 5. Faz 4 — Faturalama & ERP Entegrasyonları

Kapsam: İş İsterleri 4, 11 · Servisler: wms-billing-service, wms-integration-service

### 5.1 Çoklu Para Birimli Faturalama

- `Invoice` / `InvoiceItem` tutarları hem orijinal döviz hem yerel para karşılığıyla saklar. `InvoiceCalculationService` kur çevrimini wms-finance-service üzerinden yapar (`FinanceHttpCurrencyConversionService`); fatura numaraları `InvoiceNumberGenerator` ile üretilir.
- Rate Lock: `APPROVED` durumuna geçen faturada kur güncellemesi engellenir; ihlal girişimi `RateLockViolationException` ile reddedilir.

### 5.2 Kur Farkı Hesaplama ve Loglama

`ExchangeDifferenceService` fatura tarihi kuru ile ödeme tarihi kuru arasındaki farkı hesaplar ve `ExchangeDifferenceLog`'a kaydeder; sorgu ucu `ExchangeDifferenceController`.

### 5.3 ERP Adaptör Çerçevesi (Adapter Pattern)

- `ErpAdapter` arayüzü ve `ErpAdapterFactory` ile lokasyona atanmış ERP tipi Spring context'ten dinamik çözümlenir. Uygulanan adaptörler: `SapAdapter`, `LogoAdapter`, `MockAdapter` (test).
- Depo bazlı entegrasyon ayarları `LocationIntegrationConfig` ve `IntegrationSystem` (bağlantı tipi, yön) ile yönetilir.
- Süreç entegrasyonları: `ErpReceiptIntegrationService` (mal kabul), `ErpShipmentIntegrationService` (sevkiyat), `InventoryMovementIntegrationService` (stok hareketi).

### 5.4 Outbox Pattern ve Hata Toleransı

- İş verisiyle entegrasyon mesajı tek transaksyonda `OutboxMessage` tablosuna yazılır; `IntegrationOutboxWorker` kayıtları kilitli okuma (Pessimistic Lock) ile alır ve `OutboxMessageProcessor` asenkron gönderir.
- `OutboxRetryPolicy` Exponential Backoff uygular (üst sınır: 60 dk, konfigüre edilebilir); `maxRetry` aşımında mesaj `FAILED_MAX_RETRIES` durumuna geçer ve `OutboxAlertService` uyarı üretir.
- `IntegrationMonitorController` başarısız işleri listeleme ve Force Retry (elle yeniden deneme) uçlarını sunar; `IntegrationRetryService` süreci yürütür.
- `IntegrationArchiveScheduler` 30 günden eski `SUCCESS` entegrasyon loglarını ve `COMPLETED` outbox kayıtlarını temizler (retention konfigüre edilebilir).

## 6. Kesişen Altyapı Konuları

- Multi-tenancy: `TenantContextFilter` / `TenantContextHolder` deseni tüm servislere kopyalanarak uçtan uca tenant izolasyonu sağlandı.
- Event-driven provisioning: Yeni lokasyon açıldığında core-service Kafka'ya `LocationProvisioned` event'i yayınlar; localization, finance ve integration servisleri `LocationProvisionedConsumer` ile varsayılan konfigürasyonlarını otomatik oluşturur.
- Cache: Workflow, dinamik UI kuralları ve döviz kurları Redis üzerinde cache'lenir; konfigürasyon değişikliklerinde event tabanlı eviction uygulanır.
- Hata standardı: Tüm servislerde `GlobalExceptionHandler` + `ErrorResponse` ile tutarlı hata gövdesi; OpenAPI/Swagger dokümantasyonu (`OpenApiConfig`) her serviste aktif.
- Şema yönetimi: Flyway — core: 22, localization: 17, finance: 13, billing: 3, integration: 5 migration.

## 7. Özet Durum Tablosu

| Faz   | İş İsterleri     | Ana Çıktılar                                                                                                                                                | Durum      |
|-------|------------------|-------------------------------------------------------------------------------------------------------------------------------------------------------------|------------|
| Faz 1 | 1, 6, 7, 13      | Hiyerarşik org yapısı, ThreadLocal tenant context, Keycloak auth, UTC/timezone servisi, format API'leri, geo master data                                    | Tamamlandı |
| Faz 2 | 3, 5, 12         | Workflow Engine + AOP doğrulama, Dynamic UI Engine (alan + kolon), JSONB adres şablonları ve regex doğrulama                                                | Tamamlandı |
| Faz 3 | 8, 9, 10, 14, 15 | BigDecimal/HALF_EVEN çevrim motoru, müşteri döviz kısıtları, TCMB crawler + audit, versiyonlu vergi oranları, Strategy Pattern Tax Engine                   | Tamamlandı |
| Faz 4 | 4, 11            | Rate Lock'lu faturalama, kur farkı loglama, ERP Adapter Factory (SAP/Logo/Mock), Outbox Worker + Exponential Backoff, Force Retry API, 30 gün log temizliği | Tamamlandı |